

# Chapter 13 -- Governing Semantic Boundaries with Property Domain and Range

- [Chapter Introduction](#)
- [13.1 Why Relationship Boundaries Matter](#)
- [13.2 Understanding Property Domain and Range](#)
- [13.3 Domain - Defining Valid Relationship Sources](#)
- [13.4 Range - Defining Valid Relationships Targets](#)
- [13.5 Configuring Domain and Range in Protégé](#)
  - [13.5.1 Basic \(Single\) Domain and Range](#)
  - [13.5.2 Multi-Domain and Multi-Range Configurations](#)
    - [Modeling OR Semantics in Domain and Range](#)
      - [Options 1: Use `owl:unionOf` \(Recommended\)](#)
      - [Options 2: Create Separate Properties](#)
      - [Option 3: Use Local `SubClassOf` Restrictions](#)
    - [Summary: Choosing the Right Approach](#)
    - [Risks of Careless Multi-Domain and Multi-Range Modeling](#)
  - [13.5.3 Looking Domain/Range Configuration from EKA](#)
- [13.6 Semantic Inference Through Domain and Range](#)
- [13.7 Practical Modeling in `Pizza.owl` - Applying Domain and Range to Inverse Properties](#)
  - [13.7.1 Revisiting Inverse Properties](#)
  - [13.7.2 Applying Domain and Range to `hasTopping`](#)
  - [13.7.3 Understanding Semantic Reversal Through `isToppingOf`](#)
  - [13.7.4 Connecting Domain, Range, and Property Characteristics](#)
  - [13.7.5 Observing Reasoning Behavior in Protégé](#)
  - [13.7.6 Applying the Same Semantic Pattern to `hasBase` -- Exercise 12](#)
  - [13.7.7 Look back of Two Exercises](#)
  - [13.7.8 Why This Matters Beyond `Pizza.owl`](#)
- [13.8 EKA Tuple Mapping -- Domain and Range in Executable Knowledge Architecture](#)
  - [13.8.1  \$K\$  - Knowledge Graph](#)
  - [13.8.2  \$R\$  - Reasoning & Rules](#)
  - [13.8.3](#)

*Theta - Triggers (Future EKA Perspective)*

- 13.8.4  
*Phi* - Actions (*Future EKA Perspective*)
- 13.8.5  
*Gamma* - Governance
- 13.9 Knowledge Graph Implications -- Why Semantic Boundaries Matter
- 13.10 Governance and Semantic Integrity
- 13.11 Common Modeling Mistakes in Domain and Range
  - 13.11.1 Overly Broad Domain Definitions
  - 13.11.2 Missing Range Definitions
  - 13.11.3 Misunderstanding Multi-Domain and Multi-Range Logic
  - 13.11.4 Ignoring Inverse Property Alignment
  - 13.11.5 Treating Domain and Range Like Database Constraints
  - 13.11.6 Over-Engineering Semantic Restrictions
- 13.12 Chapter Summary
- 13.13 Exercise 12 Transition -- From Domain and Range to Semantic Restrictions
- 13.14 Key Concepts
- 13.15 Protégé Skills Learned
- 13.16 Next Chapter Preview
- 13.17 One more thing...
- Acknowledgments

## Chapter Introduction

In the previous chapter, you explored **Object Property Characteristics**, discovering that semantic relationships are not merely connections between concepts but possess behavioral meaning. Some relationships are reciprocal, some directional, some unique, and others capable of semantic propagation through inference.

Chapter (12) therefore introduced an important question:

**How should relationships behave?**

Chapter (13) now introduces another equally important question:

**Where should relationships be allowed to exist?**

This distinction is subtle, yet profoundly important.

Because in ontology engineering, semantic correctness depends not only upon:

relationship behavior,

but also:

**relationship boundaries.**

Consider a simple real-world example from `Pizza.owl`.

Suppose we define a relationship:

`hasTopping`

Intuitively, we understand that:

pizzas have toppings.

However, should:

pizza toppings have toppings?

Should:

restaurants have toppings?

Should:

customers have toppings?

Clearly, semantic meaning imposes constraints.

Relationships are not universally applicable.

They belong within:

**specific semantic contexts.**

This is precisely where:

**Property Domain and Range**

become essential!

In OWL ontology engineering:

**Domain** defines where a relationship originates. (*Source*)

and:

**Range** defines what kind of thing the relationship may point toward. (*Target* or *Destination*)

Together, they establish:

**semantic boundaries.**

Without these boundaries, ontology relationships become ambiguous.

- Reasoners lose semantic precision.
- Knowledge quality deteriorates.
- Inference becomes unreliable.

At enterprise scale, such ambiguity becomes extremely costly.

Imagine an enterprise ontology where:

Applications own Business Capabilities

and

Capabilities host Infrastructure

due to poorly governed semantic relationships.

Very quickly:

semantic chaos emerges.

Ontology engineers therefore rely on Domain and Range to preserve:

logical consistency.

Within our `Pizza.owl` tutorial, this chapter aligns with the video on:

**Property Domain and Range**

where you begin configuring these semantic restrictions directly in Protégé.

However, from the perspective of **Executable Knowledge Architecture (EKA)**, this chapter introduces something even deeper.

Recall the formal EKA tuple:

$$EKA = (K, R, \Theta, \Phi, \Gamma)$$

Property Domain and Range contribute most strongly toward:

## **$\Gamma$ - Governance**

because they define semantic validity rules.

They also strengthen:

## **$R$ - Reasoning & Rules**

because reasoners use Domain and Range information to infer class membership automatically.

Earlier chapters focused heavily on:

semantic connectivity.

This chapter introduces:

**semantic boundaries.**

And boundaries matter.

Because intelligent systems require not only rich knowledge, but also:

**trustworthy knowledge.**

This chapter therefore represents another maturity step in the `Pizza.owl` learning journey:

from:

**behavior-aware ontology**

toward:

**boundary-governed ontology.**

As you progress deeper into semantic engineering, these concepts become foundational for building:

- knowledge graphs
- enterprise ontologies
- semantic governance models
- executable architecture systems
- AI-ready knowledge structures

The deeper lesson of Chapter (13) is therefore not simply learn:

how to configure Domain and Range in Protégé.

It is learning:

**how ontology constrains meaning through semantic boundaries**

## 13.1 Why Relationship Boundaries Matter

Ontology beginners often focus on:

relationship creation.

If one concept should connect to another, an object property is created.

This feels intuitive.

After all, knowledge appears to be:

about connectivity.

However, professional ontology engineering quickly introduces another concern:

**relationship validity.**

Because merely connecting concepts is not enough.

Relationships must connect:

**the right kind of concepts.**

Otherwise, semantic meaning becomes corrupted.

Consider a simple analogy.

In natural English language, verbs imply restrictions.

For example:

a person eats food.

This sentence feels semantically correct.

But:

a building eats music

feels nonsensical.

Why?

Because semantic expectations exist.

Certain concepts logically participate in relationships.

Others do not.

Ontology formalizes this intuition through:

### **Domain and Range.**

Domain answers:

### **Who or what may use this relationship?**

Range answers:

### **What kind of thing may this relationship point to?**

These boundaries help ontology preserve:

meaning integrity.

Without semantic boundaries, ontology relationships become dangerously flexible.

Consider an enterprise example.

Suppose an ontology contains one relationship:

owns

Should:

Employee owns Application

be allowed?

Perhaps, it we're talking "Application ownership assignment".

But should:

Database owns Regulation

be valid?

Probably not!

Without semantic restrictions:

invalid knowledge spreads.

This problem becomes increasingly serious in enterprise-scale semantic systems.

Organizations may model:

- business capabilities
- processes
- regulations
- applications
- infrastructure
- vendors
- risks

Thousands of concepts become interconnected.

Without semantic boundaries:

modeling inconsistency grows easily and rapidly.

Eventually, ontology loses trustworthiness.

This challenge directly relates to:

## **Γ - Governance**

inside EKA.

Because governance is not merely about documentation.

It is about:

protecting semantic correctness.

Property Domain and Range help ontology engineers answer:

What relationships are semantically valid?

This dramatically improves the quality of ontology.



At the same time, Domain and Range strengthen:

## ***R* - Reasoning & Rules**

because ontology reasoners can infer class membership automatically.

For example in `Pizza.owl` tutorial:

If ontology defines:

`hasTopping`

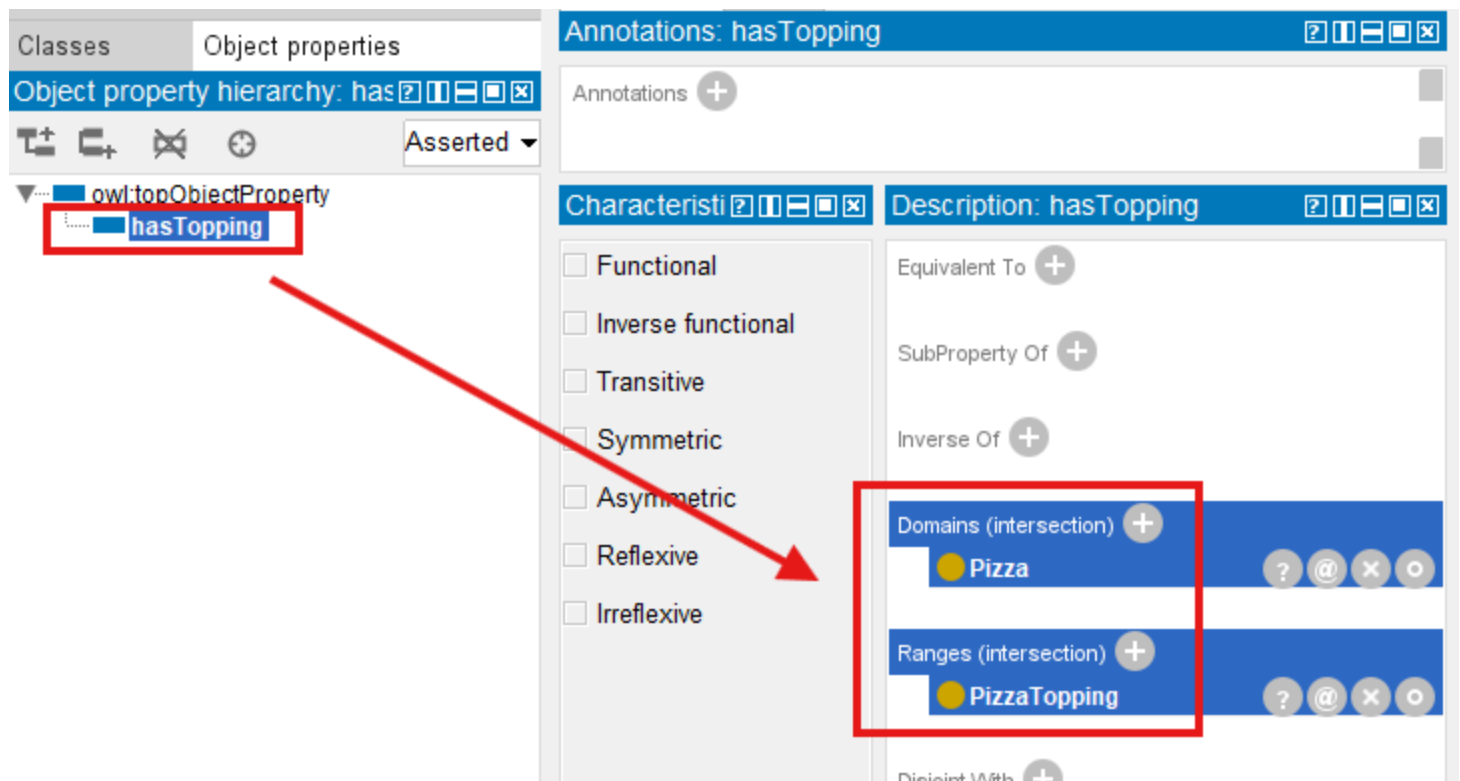
Domain:

`Pizza`

Range:

`PizzaTopping`

As below screen in Protégé:



Then if ontology later encounters:

`MySpecialPizza hasTopping MushroomTopping`

the reasoner may infer:

MySpecialPizza is a Pizza .

And:

MushroomTopping is a PizzaTopping .

Notice what happened.

Knowledge emerged:

without manual assertion.

This represents another important milestone!

Ontology is no longer merely:

representing meaning.

It is now beginning to:

**discover meaning. (new knowledge)**

## 13.2 Understanding Property Domain and Range

To understand Domain and Range deeply, you should think of object properties as:

semantic bridges.

Every bridge over a river has:

a starting point (source)

and:

a destination (target).

In ontology:

Domain = starting side

and:

Range = destination side

The Domain specifies:

which class may logically initiate the relationship.

The Range specifies:

which class may logically receive the relationship.

Consider again the `Pizza.owl` example:

`hasTopping`

This relationship connects:

`Pizza`  $\rightarrow$  `PizzaTopping`

Therefore:

| Domain             | Range                     |
|--------------------|---------------------------|
| <code>Pizza</code> | <code>PizzaTopping</code> |

Semantically, this means:

only pizzas should possess toppings.

and:

toppings should be the valid target of the relationship.

This seems obvious.

But ontology engineering benefits greatly from formalizing:

obvious meaning.

Because machines DO NOT naturally understand human assumptions.

They require:

explicit semantic rules.

This is one reason ontology engineering differs fundamentally from traditional databases.

In databases, relationships often exist through:

foreign keys.

But foreign keys typically say only:

something references something.

Ontology asks a deeper question:

**Should this relationship exist semantically?**

And:

**What meaning does this relationship imply?**

This distinction becomes especially powerful once reasoning enters the picture.

Domain and Range are not merely:

restrictions.

They also support:

**inference.**

This surprises many ontology beginners.

Because they initially assume Domain and Range simply act like validation rules.

(Luckily you're not beginner already after previous practices!)

In reality:

ontology reasoners actively use them.

Suppose ontology contains:

MyPizza hasTopping CheeseTopping

Even if:

MyPizza

has never been explicitly classified as:

Pizza ,

the reasoner may infer it automatically.

Why?

Because:

only `Pizza` (class, Domain) should participate in `hasTopping` (object property).

Similarly:

`CheeseTopping`

may become inferable as:

`PizzaTopping`.

Thus:

Domain and Range contribute to ontology intelligence.

Within EKA tuple:

$$EKA = (K, R, \Theta, \Phi, \Gamma)$$

this primarily strengthens:

**$R$  - Reasoning & Rules**

while simultaneously improving:

**$\Gamma$  - Governance**

through semantic constraint enforcement.

Ontology therefore becomes:

smarter

and

safer.

This balance matters enormously.

Because intelligence without governance becomes:

unreliable intelligence.

And governance without reasoning becomes:

rigid knowledge.

Domain and Range help maintain both.

## 13.3 Domain - Defining Valid Relationship Sources

The **Domain** of a property defines:

**where a relationship is allowed to begin.**

In simpler language:

What type of thing may logically use this property?

Let us revisit the `Pizza.owl` example:

`hasTopping`

The Domain is:

`Pizza`

This means:

pizzas may have toppings.

Ontology therefore expects:

the source entity belongs to `Pizza`.

If ontology encounters:

`Restaurant hasTopping Mushroom`

the reasoner may conclude:

`Restaurant is a Pizza` .

Clearly:

something "feels" wrong.

This illustrates an important lesson:

- reasoners trust semantic rules.

They do not understand human intention.

They only interpret:

- logical definitions.

Therefore, poorly designed domains often produce:

- unintended inference.

Professional ontology engineering therefore requires careful semantic thinking.

Always ask:

- Who should legitimately use this relationship?

Inside enterprise architecture, domain modeling becomes highly important.

Consider the relationship:

- supportedBy

Should:

- Capability supportedBy Application

be valid?

Likely yes.

But should:

- Regulation supportedBy Server

be acceptable (making sense)?

Probably not!

Domain definitions help ontology maintain:

- semantic discipline.

From an EKA perspective:

Domain modeling strengthens:

### **$\Gamma$ - Governance**

because semantic misuse becomes easier to detect.

It also supports:

### **$R$ - Reasoning & Rules**

because class membership inference improves.

This transforms ontology into something more than:

connected concepts.

Ontology now becomes:

**semantically governed knowledge.**

## **13.4 Range - Defining Valid Relationships Targets**

If Domain defines:

**where a relationship begins,**

then:

Range defines:

**where a relationship is allowed to end.**

In simpler language:

What type of thing (rdf:type) should logically receive this relationship?

Returning again to our `Pizza.owl` ontology, consider the familiar object property:

`hasTopping`

Earlier, we learned that the Domain for this property is:

`Pizza`.

However, relationships require two sides (ends).



Ontology must also understand:

what kind of thing (rdf:type) counts as a valid topping.

This is where:

## Range

becomes essential.

In `Pizza.owl` , the Range of:

`hasTopping`

is typically defined as:

`PizzaTopping` .

This means:

pizzas may point to pizza toppings.

The semantic expectation becomes clear.

A pizza should not suddenly point toward:

a restaurant

or:

a country

or:

an employee.

Instead, ontology explicitly communicates:

toppings belong to the `PizzaTopping` category.

This may initially feel restrictive.

However, semantic precision is precisely what makes ontology valuable!

Because knowledge without boundaries quickly becomes:

ambiguous knowledge.

And ambiguous knowledge often produces:

unreliable intelligence.

To illustrate this further, imagine an enterprise architecture scenario.

Suppose an ontology contains:

hostedOn

The relationship may describe:

Application hostedOn Infrastructure

In this case:

| Domain      | Range          |
|-------------|----------------|
| Application | Infrastructure |

Semantically, this means:

applications may be hosted on infrastructure.

However, ontology simultaneously prevents semantically questionable assertions such as:

Partner hostedOn Value\_Stream

or:

Business\_Capability hostedOn Employee (Business\_Actor)

These examples appear obviously INCORRECT to humans.

But semantic systems require:

explicit instruction.

Ontology engineers therefore formalize:

meaning constraints.

This becomes increasingly important when enterprise ontologies scale.

As semantic ecosystems grow larger, involving:

- objectives
- capabilities
- applications
- risks
- processes
- technologies
- implementations
- vendors
- regulations
- infrastructure

relationship quality becomes harder to maintain.

Without Range definition:

semantic inconsistency spreads quickly.

Range therefore acts as:

a semantic quality boundary.

Within EKA, Range contributes strongly toward:

**$\Gamma$  - Governance**

because it improves ontology trustworthiness.

At the same time, Range also strengthens:

**$R$  - Reasoning & Rules**

because ontology reasoners use range information to infer semantic types automatically.

Suppose ontology encounters:

SpecialPizza hasTopping Pepperoni

Even if:

Pepperoni

has not been manually classified as:

## PizzaTopping

the ontology reasoner may infer this classification automatically.

Why?

Because:

the Range of `hasTopping` defines what valid targets should be.

Let's see the quick example how inference learn from Range setting:

I have one `SpecialPizza` under `Pizza > NamedPizza`, and **wronging** create `Pepperoni` directly under `Pizza`.

From `SpecialPizza`, we create the object property `hasTopping` to `Pepperoni`, looks normal:

The screenshot displays the OntoGraf interface with the following components:

- Top Tabs:** Active ontology, Entities, Individuals by class, Individual Hierarchy Tab, DL Query, OntoGraf, Snap SPARQL.
- Class hierarchy: NamedPizza:**
  - owl:Thing
    - Pizza
      - NamedPizza (highlighted)
      - NonVegetarianPizza
      - VegetarianPizza
    - PizzaTopping

- Annotations: SpecialPizza:** (Empty list)
- Description: SpecialPizza:**
- Types: NamedPizza
- Property assertions: SpecialPizza:**
- Object property assertions: **hasTopping Pepperoni** (highlighted with a red arrow)
- Data property assertions: (+)
- Negative object property assertions: (+)
- Negative data property assertions: (+)
- Individuals:**
- Direct instances: SpecialPizza (highlighted with a red arrow)
- For: NamedPizza
- SpecialPizza (highlighted)

After synchronizing reasoner, there's no error, however, since we have Domain/Range configured for `hasTopping` property, when you switch to `Pepperoni` instance, see below:

Active ontology x Entities x Individuals by class x Individual Hierarchy Tab x DL Query x OntoGraf x Snap SPARQL

Class hierarchy: Pizza [?] [I] [H] [B] [X]

Annotations Usage

Annotations: Pepperoni [?]

Annotations +

Description: Pepperoni [?] [I] [H] [B] [X]

Property assertions: Pepperoni

Types +

Pizza ? @ x o

PizzaTopping ? @

Same Individual As +

Different Individuals +

Object property assertions +

Data property assertions +

Negative object property assertions +

Negative data property assertions +

Individuals Individuals (Inferred)

Direct instances: Pepperoni [?] [I] [H] [B] [X]

For: Pizza

Pepperoni

Although I *wrongly* put `Pepperoni` as a type of `Pizza`, now ontology semantically infer the new knowledge that `Pepperoni` is a type of `PizzaTopping`.

When you see this, it's time to think about your ontology definition and to correct / modify that.

This introduces an important realization:

Domain and Range are not merely **restrictions**.

They are also:

**semantic inference mechanisms.**

Ontology therefore becomes increasingly intelligent.

Knowledge begins organizing itself.

Reasoners begin identifying meaning.

And semantic structures become:

more explainable.

This is one of ontology engineering's greatest strengths.

## 13.5 Configuring Domain and Range in Protégé

### 13.5.1 Basic (Single) Domain and Range

After understanding the theory behind Domain and Range, you are now ready to configure these semantic constraints directly inside **Protégé**.

This chapter aligns closely with the `Pizza.owl` demonstration video, where Domain and Range are introduced through practical ontology editing (Exercise 11).

Inside Protégé, navigate to:

#### Object Properties

and select an existing property such as:

`hasTopping`

On the right-side property editing panel, you will observe dedicated sections labeled: **Domain** and **Range**.

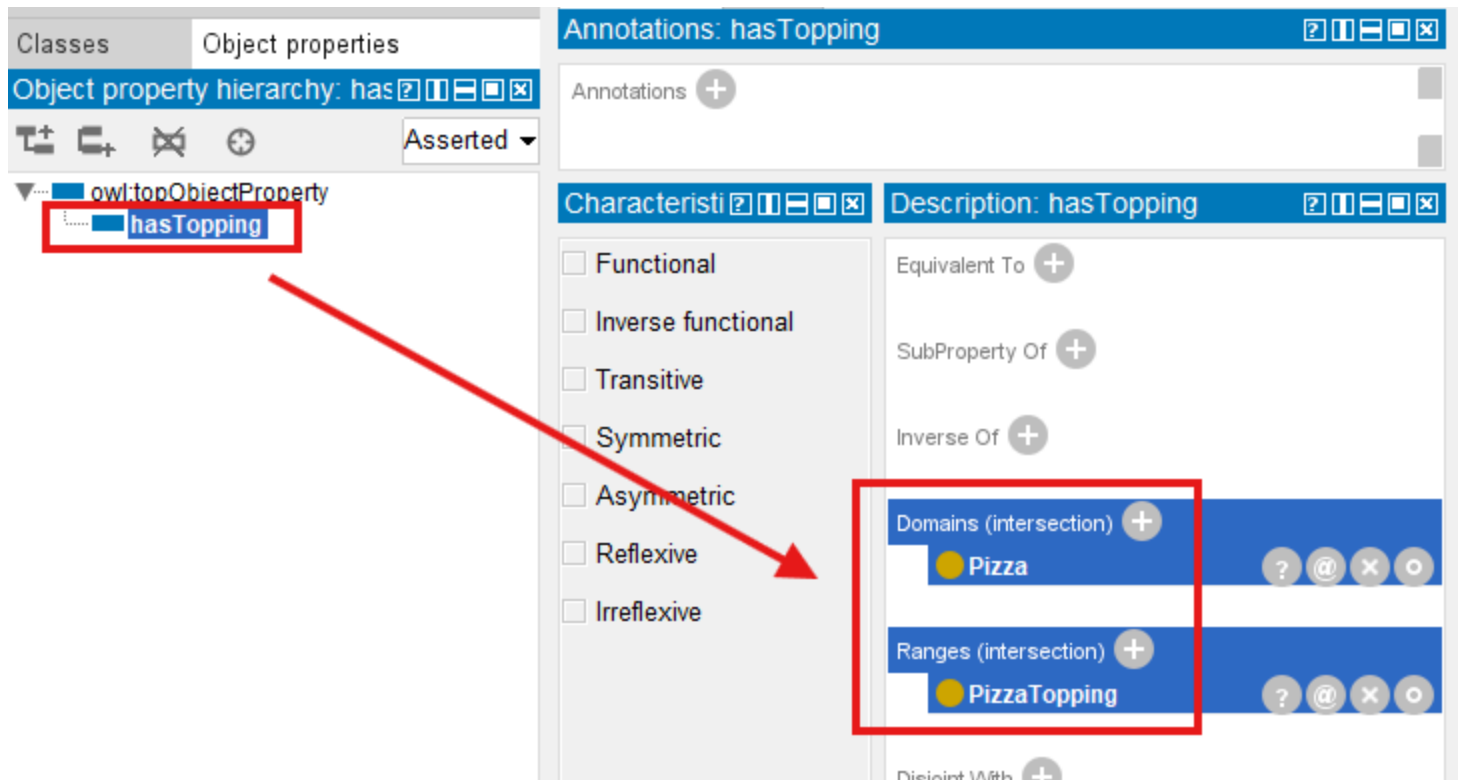
These sections allow ontology engineers to define:

semantic boundaries.

For example:

| Domain | Range        |
|--------|--------------|
| Pizza  | PizzaTopping |

The screen has been shown earlier already:



Once configured, Protégé immediately incorporates these semantic rules into the ontology model.

At this stage, you may initially perceive Domain and Range as merely:

metadata configuration.

However, they are considerably more powerful than that.

Because once the ontology reasoner is activated:

these semantic rules become operational.

The ontology now begins:

interpreting meaning.

## 13.5.2 Multi-Domain and Multi-Range Configurations

As you become more comfortable with ontology engineering, an important question naturally emerges:

Can an object property have more than one Domain or Range?

The answer is:

**Yes.**

OWL allows ontology engineers to define:

multiple Domains

and

multiple Ranges

for a single property.

At first glance, this flexibility appears highly useful.

After all, real-world enterprise relationships are rarely simple.

However, you should approach this capability carefully because:

semantic flexibility often introduces semantic complexity.

Let us first understand how this works.

Suppose an enterprise ontology contains a property:

ownedBy

An organization may wish to express:

Application ownedBy Team

while also supporting:

Business\_Capability ownedBy Team

We have common object is pointed by different subject, in such a case, ontology engineers may configure:

| Multi-Domain                                                                            | Range |
|-----------------------------------------------------------------------------------------|-------|
| <ul style="list-style-type: none"><li>Application</li><li>Business_Capability</li></ul> | Team  |

This configuration communicates:

both Applications and Business Capabilities may participate in the ownedBy relationship.

Similarly, a property may support:



multiple Ranges.

Consider:

realizedBy

An enterprise may wish to model:

Business\_Capability realizedBy Business\_Process

while also allowing (shortcut for earlier phase of building up repository):

Business\_Capability realizedBy Application

In such case:

| Domain              | Multi-Range                                                                          |
|---------------------|--------------------------------------------------------------------------------------|
| Business_Capability | <ul style="list-style-type: none"><li>Business_Process</li><li>Application</li></ul> |

Semantically, this communicates:

a capability may be realized by either a process or an application.

From a practical modeling perspective, this can be extremely useful.

Enterprise knowledge is often:

heterogeneous.

Relationships frequently span multiple semantic contexts.

Instead of creating separate properties such as:

realizedByBusinessProcess

and:

realizedByApplication

ontology engineers may reuse a single semantic relationship.

This improves:

ontology maintainability

as well as:

semantic consistency.

However, there is an important semantic consideration you must understand.

In OWL, multiple Domain or multiple Range definitions may introduce:

**intersection semantics.**

This surprises many ontology beginners.

Suppose a property declares multiple Domains:

- Pizza
- Restaurant

Ontology reasoners may interpret this as:

only entities that are both Pizza **and** Restaurant

may validly participate in the relationship.

Clearly:

this may produce unexpected inference.

Similarly, multiple Ranges may create stricter semantic behavior than originally intended.

Ontology engineers often expect:

either/or meaning.

But OWL frequently reasons using:

logical intersection.

This introduces an important lesson in ontology engineering:

reasoners follow logic, not intention.

Semantic meaning must therefore be modeled precisely.

## Modeling OR Semantics in Domain and Range

A common modeling question arises when a property legitimately needs to point to **one of several possible types** -- for example, a `realizedBy` property where the target can be either a `Business_Process` **or** `Application` (but not necessarily both).

**Important:** Adding multiple `rdfs:range` axioms will **not** achieve this. It would instead require the target to be **both** types simultaneously (intersection semantics).

To express **OR** semantics in OWL, you have several options:

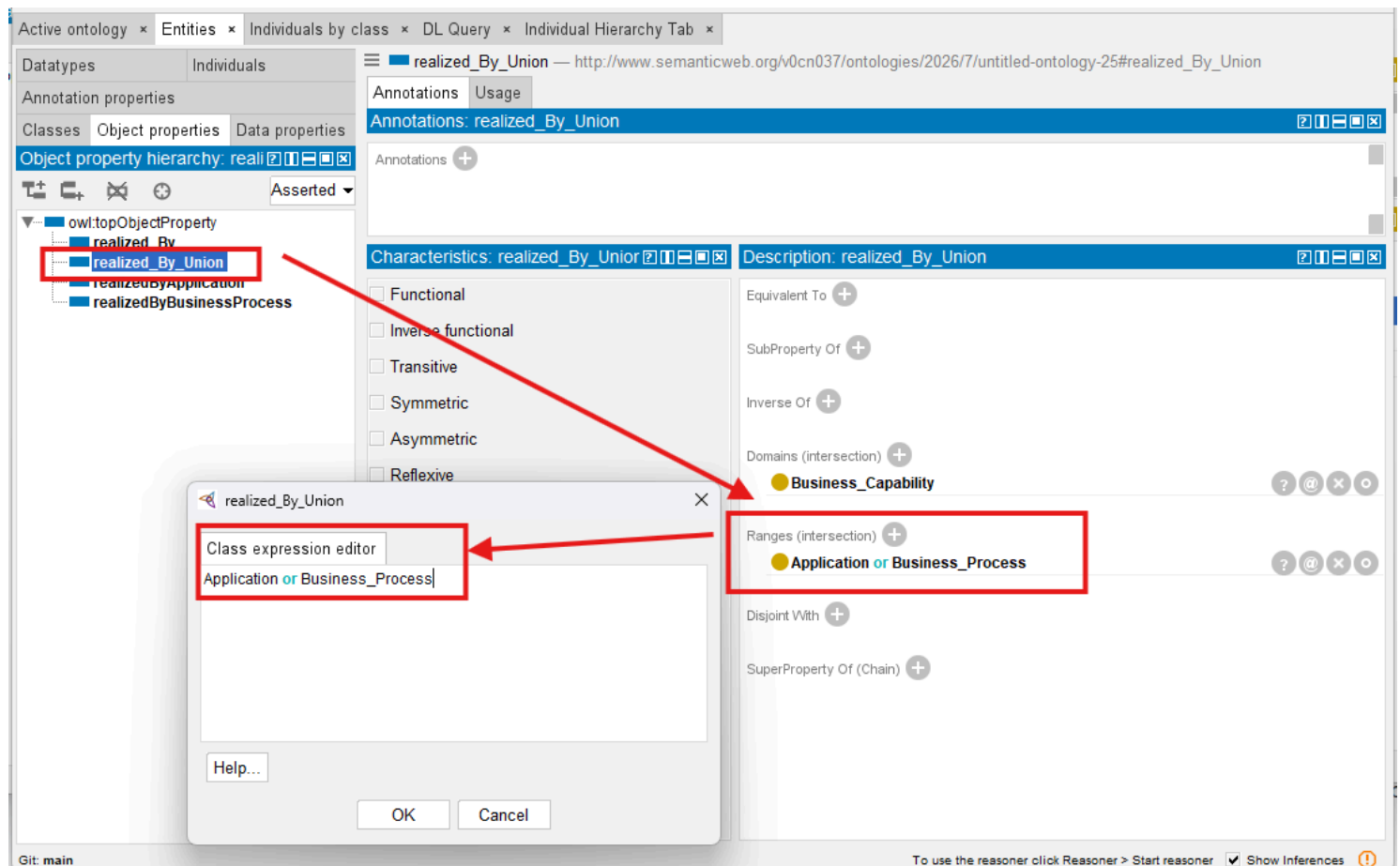
### Options 1: Use `owl:unionOf` (Recommended)

Define a union class as the range:

```
:RealizableByClass a owl:Class ;
    owl:unionOf (:Business_Process :Application)

:realized_By rdfs:range :RealizableByClass
```

In Protégé, you can enter `Business_Process` or `Application` directly in the Class Expression Editor and set it as the range, as below



I'm creating one separate working RDF file called "pizza-ebook\_13.5.2.rdf", which you can find directly in GitHub repository:

[https://github.com/yasenstar/protege\\_pizza/blob/main/ebook/rdf/pizza-ebook\\_13.5.2.rdf](https://github.com/yasenstar/protege_pizza/blob/main/ebook/rdf/pizza-ebook_13.5.2.rdf)

Inside the file, the `owl:unionOf` is applied to the `rdfs:range` within Object Properties section:

```
<!-- http://www.semanticweb.org/v0cn037/ontologies/2026/7/untitled-ontology-25#realized_By_Un:

<owl:ObjectProperty rdf:about="http://www.semanticweb.org/v0cn037/ontologies/2026/7/untitled-ontology-25#realized_By_Un:
  <rdfs:domain rdf:resource="http://www.semanticweb.org/v0cn037/ontologies/2026/7/untitled-ontology-25#realized_By_Un:
  <rdfs:range>
    <owl:Class>
      <owl:unionOf rdf:parseType="Collection">
        <rdf:Description rdf:about="http://www.semanticweb.org/v0cn037/ontologies/2026/7/untitled-ontology-25#realized_By_Un:
        <rdf:Description rdf:about="http://www.semanticweb.org/v0cn037/ontologies/2026/7/untitled-ontology-25#realized_By_Un:
      </owl:unionOf>
    </owl:Class>
  </rdfs:range>
</owl:ObjectProperty>
```

This approach is semantically clean, reusable, and tells the reasoner that the target must belong to **at least one** of the specified classes.

## Options 2: Create Separate Properties

Create `realizedByBusinessProcess` and `realizedByApplication` as separate properties.

There're pros and cons need to be aware:

- Advantage: Clearer semantics, easier to govern, no risk of unintended inference
- Disadvantage: More properties to maintain, duplicates the core semantic pattern

This is a common and acceptable pattern, especially when the relationship has different characteristics or governance requirements depending on the target type.

The screenshot shows the OntoGraf interface with the 'realizedByBusinessProcess' property selected. The left sidebar displays the 'Object property hierarchy' with 'realizedByBusinessProcess' highlighted. The main panel shows the 'Annotations' tab for this property. The 'Characteristics' section includes checkboxes for Functional, Inverse functional, Transitive, Symmetric, Asymmetric, Reflexive, and Irreflexive. The 'Description' section shows the property is 'Equivalent To' and 'SubProperty Of' other properties. The 'Domains (intersection)' section lists 'Business\_Capability' as the domain. The 'Ranges (intersection)' section lists 'Business\_Process' as the range.

The screenshot shows the OntoGraf interface with the 'realizedByApplication' property selected. The left sidebar displays the 'Object property hierarchy' with 'realizedByApplication' highlighted. The main panel shows the 'Annotations' tab for this property. The 'Characteristics' section includes checkboxes for Functional, Inverse functional, Transitive, Symmetric, Asymmetric, Reflexive, and Irreflexive. The 'Description' section shows the property is 'Equivalent To' and 'SubProperty Of' other properties. The 'Domains (intersection)' section lists 'Business\_Capability' as the domain. The 'Ranges (intersection)' section lists 'Application' as the range.

### Option 3: Use Local `subClassOf` Restrictions

Instead of a global `rdfs:range`, add local restrictions to the domain class:

Business\_Capability

SubClassOf realizedBy some Business\_Process

SubClassOf realizedBy some Application

Data properties

Annotation properties

Classes

Object properties

Class hierarchy: Business\_C

owl:Thing

Application

Business\_Capability

Business\_Process

Asserted

Annotations

Usage

Annotations: Business\_Capability

Annotations +

Description: Business\_Capability

Equivalent To +

SubClass Of +

realizedBy some Application

realizedBy some Business\_Process

General class axioms +

This pops up the scope comparison between **Global** and **Local**:

| Approach                    | Scope                                                  | Inference Behavior                |
|-----------------------------|--------------------------------------------------------|-----------------------------------|
| <code>rdfs:range</code>     | <b>Global</b> — applies to every use of the property   | Inferred for all subjects         |
| SubClassOf some restriction | <b>Local</b> — applies only to instances of that class | Inferred only for those instances |

This approach avoids global side effects and allows different target types for different subclasses. It is a recommended practice in many ontology engineering circles.

Another working RDF "pizza-ebook\_13.5.2\_2.rdf" is created for your to view the model source:  
[https://github.com/yasenstar/protege\\_pizza/blob/main/ebook/rdf/pizza-ebook\\_13.5.2\\_2.rdf](https://github.com/yasenstar/protege_pizza/blob/main/ebook/rdf/pizza-ebook_13.5.2_2.rdf)

These **Local** restrictions are appeared in the "Classes" section:

```
<!-- http://www.semanticweb.org/v0cn037/ontologies/2026/7/untitled-ontology-26#Business_Capab:

<owl:Class rdf:about="http://www.semanticweb.org/v0cn037/ontologies/2026/7/untitled-ontology-:
  <rdfs:subClassOf>
    <owl:Restriction>
      <owl:onProperty rdf:resource="http://www.semanticweb.org/v0cn037/ontologies/2026/:
      <owl:someValuesFrom rdf:resource="http://www.semanticweb.org/v0cn037/ontologies/20
    </owl:Restriction>
  </rdfs:subClassOf>
</rdfs:subClassOf>
  <owl:Restriction>
    <owl:onProperty rdf:resource="http://www.semanticweb.org/v0cn037/ontologies/2026/:
    <owl:someValuesFrom rdf:resource="http://www.semanticweb.org/v0cn037/ontologies/20
  </owl:Restriction>
</rdfs:subClassOf>
</owl:Class>
```

### Summary: Choosing the Right Approach

| Approach                                         | When to Use                                                                                            |
|--------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| <b>owl:unionOf range</b>                         | The property is semantically the same but can point to different types; good for reuse and consistency |
| <b>Separate properties</b>                       | The relationship has different characteristics or governance requirements for each target type         |
| <b>Local restrictions</b><br>( SubClassOf some ) | You want to avoid global side effects; excellent for modular ontologies                                |
| <b>Multiple rdfs:range axioms</b>                | <b>Avoid</b> — because of the unintended AND /intersection semantics                                   |

### Risks of Careless Multi-Domain and Multi-Range Modeling

Even with the OR patterns available, ontology engineers must still exercise care when using multi-Domain and multi-Range configuration

Within enterprise ontology, **careless use** of multi-Domain and multi-Range modeling may introduce:

- unintended class inference
- semantic ambiguity
- governance complexity

- difficult-to-explain reasoning outcomes

This becomes especially important in:

enterprise-scale semantic systems

where ontology quality directly influences:

knowledge graph reliability.

### 13.5.3 Looking Domain/Range Configuration from EKA

From the perspective of:

$$EKA = (K, R, \Theta, \Phi, \Gamma)$$

overly flexible semantic boundaries may weaken:

*Gamma* - **Governance**

because semantic correctness becomes harder to validate.

It may also complicate:

*R* - **Reasoning & Rules**

because reasoning outcomes become increasingly difficult to predict and explain.

For this reason, a useful practical recommendation for ontology beginners is:

**start simple before becoming flexible!**

Begin with single Domain and single Range definitions whenever possible.

Only introduce multi-Domain or multi-Range when clear semantic meaning genuinely requires it.

A good ontology engineering principle is:

**semantic simplicity before semantic flexibility.**

This keeps ontology:

- easier to understand,
- easier to govern, and
- easier to reason about.



As ontology maturity grows, multi-domain/multi-range modeling becomes increasingly valuable for representing:

- real-world enterprise complexity.

Returning to the `Pizza.owl` tutorial, you may notice that `Pizza.owl` intentionally uses:

- relatively precise semantic boundaries.

This design choice is deliberate.

The tutorial aims to help you first understand:

- semantic correctness

before introducing:

- semantic flexibility.

Once you master foundational concepts, more advanced modeling strategies become easier to apply responsibly.

For example, support an individual exists:

- `MyDinner`

and ontology contains the statement:

- `MyDinner hasTopping MozzarellaTopping`

Even if:

- `MyDinner`

has never been manually asserted as:

- `Pizza` ,

the reasoner may infer:

- `MyDinner rdf:type Pizza`.

Similarly:

- `MozzarellaTopping`

may automatically become inferable as:

PizzaTopping .

This moment is particularly important for you.

Because ontology suddenly demonstrates something databases rarely achieve naturally:

### **semantic type discovery.**

Traditional systems often require explicit classification.

Ontology reasoners can infer classification based upon:

relationship behavior.

This is a major milestone in semantic engineering.

Because ontology transitions from:

manually structured knowledge

toward:

### **self-organizing semantic intelligence.**

Within the EKA perspective:

$$EKA = (K, R, \Theta, \Phi, \Gamma)$$

Protégé configuration of Domain and Range primarily strengthens:

### **$R$ - Reasoning & Rules**

because semantic logic now influences inference behavior.

However, it also reinforces:

### **$\Gamma$ - Governance**

because semantic misuse becomes more visible.

Poorly defined relationships quickly reveal themselves through:

unintended inference.

Ontology therefore teaches an important engineering lesson:

semantic precision matters.

## 13.6 Semantic Inference Through Domain and Range

One of the most powerful yet often overlooked aspects of Domain and Range is their ability to support:

**automatic semantic inference.**

Many beginners initially assume:

Domain and Range only validate relationships.

In reality:

ontology reasoners actively use them to discover meaning.

This fundamentally changes how knowledge behaves.

Consider again our `Pizza.owl` example:

`MyCustomPizza` `hasTopping` `OliveTopping`

Suppose neither `MyCustomPizza` nor `OliveTopping` has explicit class definitions.

In Protégé, technically, you may put them as instance of a dummy class e.g. `NoClassDefined`, as below:

The screenshot displays the Protégé ontology editor interface. The top-left pane shows the 'Class hierarchy' for 'NoClassDefined', which includes 'owl:Thing', 'NoClassDefined', 'Pizza', and 'PizzaTopping'. The top-right pane shows 'Annotations: MyCustomPizza'. The bottom-left pane shows 'Individuals (Inferred)' with a list of instances: 'NoClassDefined', 'MyCustomPizza', and 'OliveTopping'. The bottom-right pane shows 'Property assertions: MyCustomPizza' with a list of assertions: 'hasTopping OliveTopping'. Red boxes highlight the 'NoClassDefined' class in the hierarchy, the 'NoClassDefined' instance in the individuals list, and the 'hasTopping OliveTopping' assertion in the property assertions list.

Then if ontology defines:

| Domain | Range        |
|--------|--------------|
| Pizza  | PizzaTopping |

then a reasoner may infer:

MyCustomPizza is a Pizza

and

OliveTopping is a PizzaTopping .

While, notice here we use "**may**", in actual sample, the subject is not inferred, but semantically it's already good enough for now.

### A note about Protégé display:

Under the OWL semantics of `rdfs:domain` , the subject of `hasTopping` should also be inferred as a `Pizza` . If the subject is not shown as inferred in your Protégé environment, this should not be interpreted as meaning that domain inference works **ONLY** in one direction.

The displayed result may depend on the asserted type of the individual, the active reasoner, and the reasoner configuration.

In this example, the `NoClassDefined` assertion and the Protégé reasoning view affect what is visibly presented.

The important semantic rule remains: **a property assertion can imply the type of its subject through the property's domain.**

**Review note:** This clarification was added following a technical review by **Timothy W. Cook**, who noted the potential ambiguity between the OWL semantic consequence and what a learner may see displayed in Protégé.

- **Observed Protégé display -- subject inference not visibly shown:**

Active ontology x Entities x Individuals by class x Individual Hierarchy Tab x DL Query x OntoGraf x Snap SPARQL x

Class hierarchy: NoClassDefi [?] [I] [E] [B] [X]

Annotations Usage

Annotations: MyCustomPizza

Annotations +

Individuals Individuals (Inferred)

Direct instances: MyCustomF [?] [I] [E] [B] [X]

Types +

NoClassDefined

For: NoClassDefined

MyCustomPizza

OliveTopping

Description: MyCustomPizza [?] [I] [E] [B] [X]

Property assertions: MyCustomPizza

Object property assertions +

hasTopping OliveTopping

Data property assertions +

Negative object property assertions +

Negative data property assertions +

• Observed Protégé display -- object inference shown:

Active ontology x Entities x Individuals by class x Individual Hierarchy Tab x DL Query x OntoGraf x Snap SPARQL x

Class hierarchy: NoClassDefi [?] [I] [E] [B] [X]

Annotations Usage

Annotations: OliveTopping

Annotations +

Individuals Individuals (Inferred)

Direct instances: OliveToppir [?] [I] [E] [B] [X]

Types +

NoClassDefined

PizzaTopping

For: NoClassDefined

MyCustomPizza

OliveTopping

Description: OliveTopping [?] [I] [E] [B] [X]

Property assertions: OliveTopping

Object property assertions +

Data property assertions +

Negative object property assertions +

Negative data property assertions +

This appears simple.

Yet conceptually:

it is revolutionary.

Because semantic classification emerges:

automatically.

Ontology no longer depends solely on manual labeling.

Instead:

meaning becomes inferable.

This capability becomes enormously important within enterprise environments.

Imagine an enterprise ontology describing:

CRM\_Application hostedOn Cloud\_Platform

If:

| Domain      | Range          |
|-------------|----------------|
| Application | Infrastructure |

then ontology may infer:

CRM\_Application is an ( `rdf:type` ) Application

and

Cloud\_Platform is ( `rdf:type` ) Infrastructure.

At enterprise scale, where organizations manage:

thousands or millions of entities,

automatic inference dramatically reduces:

modeling effort.

More importantly:

it improves consistency.

Humans frequently forget classification.

Ontology reasoners DO NOT.

This makes semantic systems significantly more resilient than traditional documentation approaches.

Within EKA tuple:

$$EKA = (K, R, \Theta, \Phi, \Gamma)$$

semantic inference through Domain and Range strongly enhances:

### ***R* - Reasoning & Rules**

because knowledge expands automatically.

It also improves:

### ***K* - Knowledge Graph**

because inferred classifications enrich graph meaning.

Knowledge graphs become increasingly valuable when semantic meaning is not only stored -- but:

**inferred dynamically.**

This creates an important bridge between:

ontology

and

executable knowledge systems.

Because intelligence emerges not merely from data availability -- but from:

**machine-understandable semantics.**

This is precisely why ontology remains foundational to the long-term vision of EKA:

transforming enterprise architecture into executable intelligence.

## **13.7 Practical Modeling in `Pizza.owl` - Applying Domain and Range to Inverse Properties**

At this stage of the ontology learning journey, you may naturally begin asking an important question:

What happens when Domain and Range are applied to inverse properties?

This question becomes particularly important because the `Pizza.owl` tutorial does not introduce Domain and Range in isolation.

Instead, within **Exercise 11**, Michael demonstrates these semantic boundaries using:

**an inverse property pair.**

This is highly meaningful from an ontology engineering perspective.

Because learners are no longer simply defining:

semantic relationships.

They are now beginning to understand how:

multiple semantic mechanisms work together.

This chapter therefore acts as a convergence point among:

- **Chapter 11: Inverse Properties**
- **Chapter 12: Object Property Characteristics**
- **Chapter 13: Domain and Range**

Ontology engineering now starts becoming:

interconnected semantic modeling.

## 13.7.1 Revisiting Inverse Properties

In Chapter (11), you explored:

**inverse properties**

through familiar `Pizza.owl` examples such as:

`hasTopping`

and:

`isToppingOf`

These two properties represent:

opposite semantic directions.



If:

Pizza hasTopping MushroomTopping

then ontology can infer:

MushroomTopping isToppingOf Pizza

This helped you understand:

semantic bidirectionality.

However, in Exercise 11, Michael extends this idea further.

The ontology now introduces **Domain** and **Range** into this inverse relationship pair.

This creates a much richer semantic model.

### 13.7.2 Applying Domain and Range to hasTopping

Consider the object property:

hasTopping

The tutorial typically defines:

| Domain | Range        |
|--------|--------------|
| Pizza  | PizzaTopping |

This means:

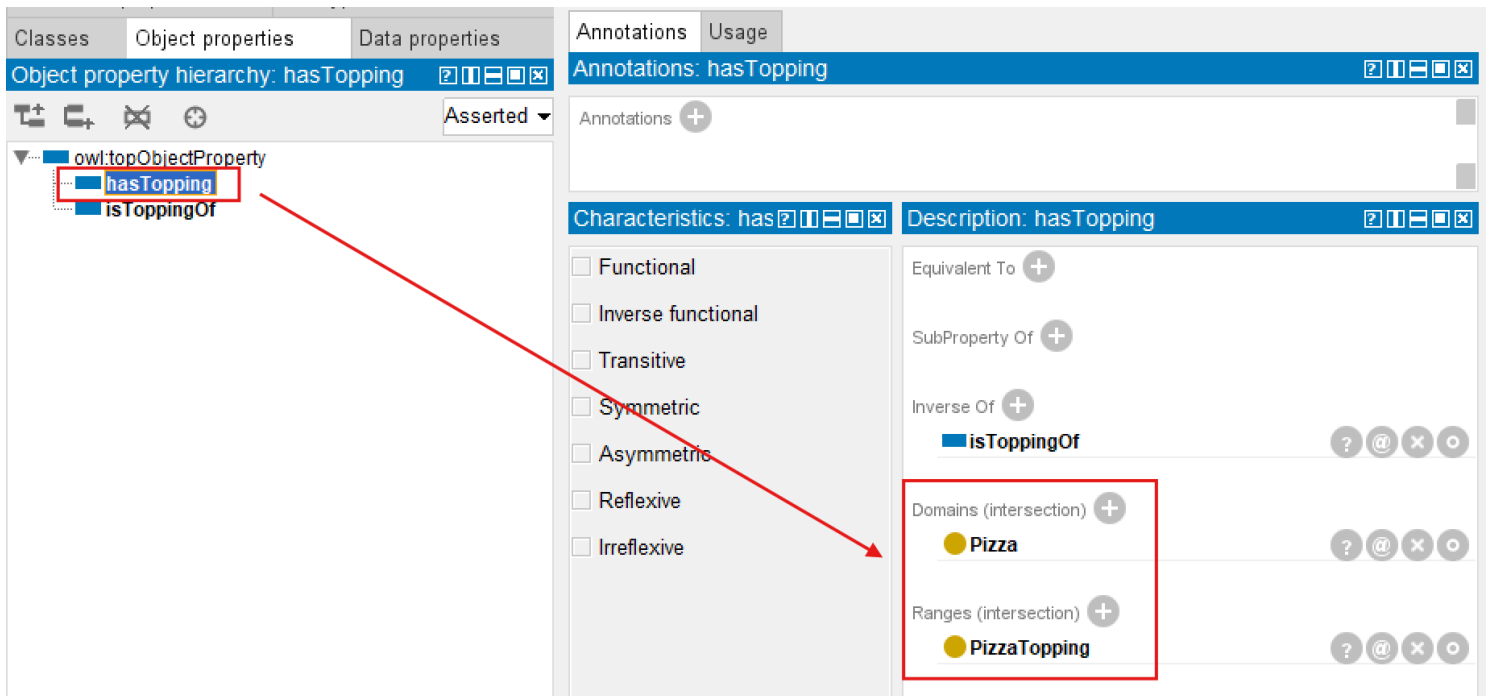
pizzas may possess toppings.

Semantically:

the relationship originates from Pizza

and points toward:

PizzaTopping



At first glance, this appears straightforward.

However, ontology meaning becomes much more interesting once inverse properties are considered.

### 13.7.3 Understanding Semantic Reversal Through `isToppingOf`

Because:

`isToppingOf`

is defined as the inverse of:

`hasTopping`,

see above screen, the semantic direction reverses.

Ontology therefore naturally expects:

| Domain       | Range |
|--------------|-------|
| PizzaTopping | Pizza |

The screenshot shows an ontology editor interface. On the left, under 'Object property hierarchy: isToppingOf', the property 'isToppingOf' is selected. A red arrow points from this selection to the 'Description: isToppingOf' pane on the right. In this pane, the 'hasTopping' property is listed under 'Inverse Of'. Below it, the 'Domains (intersection)' and 'Ranges (intersection)' sections show 'PizzaTopping' and 'Pizza' respectively, both highlighted in light yellow to indicate they are inferred by the reasoner.

The light yellow colors behind indicates they are inferred by reasoner.

This subtle reversal teaches an important ontology lesson:

inverse relationships reverse semantic boundaries.

In other words:

If:

$\text{Pizza} \rightarrow \text{PizzaTopping}$

through:

`hasTopping`,

then ontology logically expects:

$\text{PizzaTopping} \rightarrow \text{Pizza}$

through:

`isToppingOf`.

This demonstrates something profoundly important:

semantic constraints themselves become reversible.

Ontology reasoning therefore behaves consistently across:

bidirectional meaning.

This is one of the elegant strengths of OWL modeling.

Because semantic logic remains:

coherent.

Relationships do not merely point in opposite directions.

They preserve:

semantic validity.

## 13.7.4 Connecting Domain, Range, and Property Characteristics

This exercise also quietly reinforces ideas introduced in:

### Chapter (12) -- Object Property Characteristics

You previously discovered that properties may possess:

- inverse relationships
- uniqueness constraints
- transitive behavior
- symmetric meaning

Now, Chapter (13) introduces another insight:

semantic boundaries also influence property behavior.

Ontology relationships therefore consist of multiple semantic layers:

- Relationship Meaning: What does this property represent?
- Relationship Direction: Can meaning flow both ways?
- Relationship Behavior: How does reasoning behave?
- Relationship Boundary: Where should the relationship logically exist?

This layered perspective represents an important maturity step.

Because ontology engineering is no longer simply:

connecting concepts.

It becomes:

governing semantic behavior.

In `Pizza.owl` , you begin seeing how:

inverse properties

and

Domain/Range definitions

cooperate to maintain:

semantic consistency.

Without such coordination, reasoning outcomes would become:

contradictory.

## 13.7.5 Observing Reasoning Behavior in Protégé

Now, you're encouraged to experiment direction in Protégé.

Select `hasTopping` , and observe:

| Domain             | Range                     |
|--------------------|---------------------------|
| <code>Pizza</code> | <code>PizzaTopping</code> |

Then inspect `isToppingOf` and notice the semantic reversal.

Ontology now communicates a complete semantic story:

pizza has toppings

and:

toppings belongs to pizzas.

More importantly:

ontology understands this **automatically**.

Suppose ontology encounters:

`Pepperoni isToppingOf MySpecialPizza`

Even if neither entity has explicit classification, the reasoner may infer:

```
Pepperoni is a PizzaTopping
```

and:

```
MySpecialPizza is a Pizza .
```

This is particularly powerful!

Because ontology begins:

```
discovering meaning through relationships.
```

Instead of relying entirely upon manual classification, semantic inference emerges through:

```
logical structure.
```

This is one reason ontology engineering differs profoundly from traditional databases.

Databases generally require **explicit typing**.

Ontology can infer **semantic identity**.

## 13.7.6 Applying the Same Semantic Pattern to `hasBase` -- Exercise 12

After you complete **Exercise 11**, Michael immediately reinforces the same modeling technique through:

### **Exercise 12 -- Define the Domain and Range for the `hasBase` Property**

At first glance, this may appear repetitive.

However, from an ontology engineering perspective, this repetition is highly intentional.

Because semantic modeling skills improve not by learning isolated examples -- but through:

```
repeated semantic patterns.
```

In Exercise 11, you configured `hasTopping` and observed how the inverse property `isToppingOf` automatically inherited corresponding semantic boundaries through reasoning.

Exercise 12 now applies the same semantic logic to a different relationship `hasBase` .

This helps you recognize an important ontology principle:

semantic governance should be consistently reusable.

Inside Protégé, you repeat a familiar modeling sequence.

First, navigate to the **"Object Properties"** tab and select `hasBase`.

Next, configure the **Domain** by clicking the (+) Add icon next to **Domains (intersection)** inside the Description view.

From the Class Hierarchy, select `Pizza`.

This establishes an important semantic rule:

only pizzas should logically have a pizza base.

The relationship therefore gains: semantic charity.

Next, repeat the process for **Range**.

Again using the (+) Add icon next to **Ranges (intersection)**, select `PizzaBase`.

This establishes another important semantic boundary:

pizza may only connect to valid pizza bases.

At this point, you should synchronize the reasoner.

Now select the inverse property `isBaseOf`, as below screen:

The screenshot displays the Protégé interface with the 'Object properties' tab selected. The 'Object property hierarchy: isBaseOf' panel on the left shows a tree structure with 'isBaseOf' selected. The 'Annotations: isBaseOf' panel is empty. The 'Characteristics: isB:' panel shows various checkboxes for property characteristics, all of which are currently unchecked. The 'Description: isBaseOf' panel shows the configuration for the property's domain and range. Under 'Domains (intersection)', the class 'PizzaBase' is selected. Under 'Ranges (intersection)', the class 'Pizza' is selected. The 'Disjoint With' section is also visible.

A particularly important observation now emerges.

Rather than manually configuring semantic boundaries again, you will see that:

the reasoner automatically fills the Domain and Range.

Ontology now infers:

| Property | Domain    | Range |
|----------|-----------|-------|
| isBaseOf | PizzaBase | Pizza |

This mirrors the semantic reversal observed earlier with `hasTopping` and `isToppingOf`.

The significance of this exercise should not be underestimated.

Because you are now witnessing an important ontology engineering pattern:

semantic boundaries propagate through inverse relationships.

This dramatically reduces modeling effort while simultaneously improving:

- semantic consistency
- ontology maintainability
- reasoning reliability

Instead of repeatedly configuring mirrored semantics manually, ontology can increasingly:

govern itself through logical structure.

This becomes especially important in enterprise-scale ontology engineering, where hundreds or even thousands of semantic relationships may exist.

Exercise 12 therefore reinforces an important professional modeling habit:

whenever a new object property is introduced, carefully consider its Domain, Range, and inverse semantic behavior.

Ontology engineering is gradually shifting from:

manually connected concepts

toward:

**governed semantic systems.**



## 13.7.7 Look back of Two Exercises

Together, Exercises 11 and 12 quietly introduce another important realization.

Ontology relationships are not isolated configurations.

Instead, semantic modeling follows: **reusable patterns**.

The same logic applied to `hasTopping` can later be applied to `hasBase`, and eventually to much larger enterprise semantic relationships.

This progression matters because ontology engineering increasingly depends on:

- semantic consistency at scale.

As you move toward the next chapter, they will begin discovering that semantic boundaries alone are not enough.

Ontology must also express **semantic requirements**.

This transition introduces the next major concept in OWL modeling:

### **Restrictions.**

Because saying:

- `Pizza hasBase PizzaBase`

describes:

- allowable relationships.

But saying:

- every `ThinAndCrispyPizza` must have `ThinAndCrispyBase`

introduces:

- semantic logic.

And this marks another important maturity step toward:

### **Executable Knowledge Architecture (EKA)**

# 13.7.8 Why This Matters Beyond Pizza.owl

Although the `Pizza` ontology uses intentionally simple examples, the modeling pattern introduced here becomes highly valuable at:

enterprise scale.

Consider enterprise relationships such as:

Application realizes Capability

The inverse property may be:

Capability realizedBy Application

Domain and Range immediately become important.

Semantic correctness requires:

## realizes

| Domain      | Range      |
|-------------|------------|
| Application | Capability |

## realizedBy

| Domain     | Range       |
|------------|-------------|
| Capability | Application |

Notice how:

semantic reversal preserves logical meaning.

This becomes essential for:

- dependency analysis
- impact tracing
- governance validation
- knowledge graph navigation

Ontology therefore evolves beyond:

static modeling (diagramming).

It becomes:

**explainable semantic intelligence.**

From the EKA perspective of:

$$EKA = (K, R, \Theta, \Phi, \Gamma)$$

Exercise 11 strengthens:

**$R$  - Reasoning & Rules**

through semantic inference, while simultaneously reinforcing:

**$\Gamma$  - Governance**

through controlled semantic boundaries, also provide flexibility to:

**$K$  - Knowledge Graph**

through automatically adding possible inferred relationships directions.

This chapter therefore represents another important maturity transition:

from:

semantic relationships

toward:

**semantically governed relationships.**

## 13.8 EKA Tuple Mapping -- Domain and Range in Executable Knowledge Architecture

At first glance, Domain and Range may appear to be relatively small ontology configurations.

Within Protégé, they are represented as:

simple semantic fields.

Ontology engineers merely select:

- a Domain

and/or:

- a Range.

However, beneath this apparent simplicity lies something much deeper.

From the perspective of:

$$EKA = (K, R, \Theta, \Phi, \Gamma)$$

Domain and Range represent an important transition point where ontology evolves from:

- connected semantic concepts

toward:

- governed executable semantics.**

This chapter therefore strengthens multiple EKA components simultaneously.

## 13.8.1 $K$ - Knowledge Graph

The first major contribution of Domain and Range appears within:

- $K$  - Knowledge Graph.**

Earlier chapters introduced semantic relationships through:

- object properties.

However, simply connecting concepts DOES NOT automatically produce:

- meaningful knowledge.

Relationships without boundaries quickly become:

- semantically noisy.

At enterprise scale, poorly governed semantic relationships often lead to:

- graph pollution (or someone calls "graph poisoning").

Business capabilities reference unrelated entities.

Applications connect to inappropriate infrastructure.

Processes point toward semantically incompatible concepts.

Over time:

knowledge quality degrades.

Domain and Range help prevent this problem.

They establish:

semantic relationships boundaries.

In effect, ontology introduces:

**graph discipline!**

When ontology is later transformed into graph technologies such as **Neo4j**, semantic constraints significantly improve:

- graph consistency
- traversal accuracy
- query quality
- explainability

This becomes increasingly important in enterprise architecture because organizations typically maintain:

highly interconnected semantic ecosystems.

Without clear boundaries:

graph meaning becomes difficult to trust.

Domain and Range therefore help ensure that:

connected data becomes meaningful knowledge.

This represents a critical maturity step toward:

**knowledge-driven architecture intelligence.**

## 13.8.2 *R* - Reasoning & Rules

Perhaps the strongest contribution of this chapter appears within:

## ***R* - Reasoning & Rules.**

As demonstrated in Exercise 11, reasoners actively use Domain and Range for:

- semantic inference.

This is often surprising to ontology beginners.

Many learners initially assume Domain and Range merely function as:

- validation rules.

However, ontology reasoners interpret them as:

- semantic logic!

Support ontology contains:

- Pepperoni isToppingOf MySpecialPizza

Through inverse properties and semantic boundaries, the ontology reasoner may infer:

- Pepperoni is a PizzaTopping

and:

- MySpecialPizza is a Pizza.

Notice something important: **ontology inferred meaning.**

No manual classification was required.

Instead:

- semantic relationships generated semantic understanding.

This becomes profoundly important in enterprise systems.

Imagine a large enterprise ontology containing:

- thousands of applications, processes, platforms, risk, and capabilities.

Manual semantic maintenance quickly becomes: **unsustainable.**

Reasoners therefore become increasingly valuable.

Ontology shifts from:

manually curated knowledge

toward:

**self-organizing semantic intelligence.**

Within EKA, this directly strengthens:

***R* - Reasoning & Rules**

because semantic meaning becomes:

machine-discoverable.

This capability later supports:

- impact analysis
- dependency tracing
- automated classification
- semantic consistency validation

In many ways, this chapter quietly strengthens the foundation of:

**executable intelligence.**

### 13.8.3 $\Theta$ - Triggers (*Future EKA Perspective*)

Although `Pizza.owl` does not yet implement semantic triggers directly, Domain and Range provide important groundwork for:

**$\Theta$  - Triggers.**

In future enterprise semantic systems, invalid semantic relationships may automatically initiate:

governance responses.

For example:

Suppose an enterprise ontology accidentally contains:

`Compliance_Policy` hosted on Database

If ontology semantics define `hosted on` with:

| Domain      | Range          |
|-------------|----------------|
| Application | Infrastructure |

then semantic inconsistency may be detected automatically.

Future EKA systems could trigger:

- semantic quality checks
- governance alerts
- architecture review workflows
- validation dashboards

Ontology therefore begins moving beyond:

passive modeling

toward:

**active semantic monitoring.**

This is one of the long-term ambitions of EKA:

transforming knowledge into executable capability.

## 13.8.4 $\Phi$ - Actions (*Future EKA Perspective*)

Once semantic triggers emerges, the next logical step becomes:

**$\Phi$  - Actions**

Reasoning outcomes may eventually support:

automated execution.

For example:

When invalid semantic relationships are discovered/detected, enterprise systems might automatically:

- notify enterprise architects
- log inconsistency tracking registry
- generate remediation tasks
- update architecture repositories
- trigger change governance processes



Ontology therefore evolves beyond:

| descriptive modeling

toward:

| **operational semantic intelligence.**

Although Chapter 13 remains foundational, it quietly introduces important prerequisites for future executable architecture systems.

## 13.8.5 $\Gamma$ - Governance

Perhaps most importantly, Domain and Range strongly reinforce:

|  **$\Gamma$  - Governance.**

Because governance ultimately depends on:

| semantic trustworthiness.

Without boundaries: semantic quality declines --

- Reasoning becomes unreliable.
- Knowledge graphs become polluted.
- Business confidence weakens.

Ontology therefore requires:

| **semantic guardrails.**

Domain and Range provide an important part of this capability.

They help ontology engineers answer:

| What relationships are semantically valid?

More importantly:

| What relationships should not exist?

This distinction matters enormously!

Because ontology engineering is not merely about:

what is possible.

It is more about:

**what is meaningful!**

Domain and Range therefore represent another important maturity step in EKA:

from:

semantic connectivity

toward:

**semantic governance.**

At this point, however, an important distinction must be made between **semantic inference** and **governance enforcement**.

Domain and Range provide semantic guardrails by defining how relationships are interpreted within the ontology. When a reasoner processes these axioms, it can derive additional knowledge from them. In this sense, Domain and Range contribute directly to **machine-interpretable semantic governance**.

However, OWL inference should not be confused with operational enforcement:

**OWL inference determines what the ontology logically implies;**

**Governance enforcement determines what a governed system is permitted to accept or act upon.**

An OWL reasoner does not necessarily reject a statement simply because it leads to an unexpected or undesirable semantic consequence. As demonstrated earlier in this chapter, the ontology may instead interpret the statement according to its axioms and derive additional knowledge.

Therefore, a complete governance architecture may involve several complementary mechanisms:

- **Ontology semantics** define intended meaning.
- **Reasoning** derives logical consequences.
- **Validation** evaluates whether data conforms to defined constraints.
- **Policy enforcement** determines whether an operation or decision should be accepted, rejected, flagged, or remediated.

Domain and Range should therefore be understood as an important **semantic governance foundation**, rather than as a complete enforcement mechanism.



Note

## Reviewer's Insight — Inference Is Not Enforcement

This distinction was strengthened following a technical review by **Timothy W. Cook**, Founder and CEO of Axius SDC, Inc.

His review highlighted an important engineering distinction: OWL reasoning determines what the ontology logically implies, while validation and policy-enforcement mechanisms determine what a governed system should accept, reject, flag, or remediate.

This observation helped sharpen the interpretation of  $\Gamma$  — **Governance** within the EKA framework and will continue to inform the governance and validation discussions in later stages of the Semantic Knowledge Development Lifecycle.

At the level of  $K$ , knowledge is represented.

Through  $R$ , relationships and their logical consequences become interpretable and inferable.

Through  $\Gamma$ , the semantic knowledge becomes governed through mechanisms for semantic discipline, validation, and, where required, enforcement.

Later,  $\Theta$  and  $\Phi$  can connect governed semantic conditions to triggers and actions.

The progression can therefore be viewed as:

$$K/R \rightarrow \Gamma \rightarrow \Theta/\Phi$$

where  $\Gamma$  serves as the important bridge between **semantic interpretation** and **controlled operational use**.

This is why governance should not be understood simply as another collection of OWL axioms. It represents the broader engineering capability required to ensure that semantic knowledge remains trustworthy when it moves from an ontology model into real-world information systems and decisions.

## 13.9 Knowledge Graph Implications -- Why Semantic Boundaries Matter

In Chapter (08), you explored RDF as:

the structural language of ontology.

At that time, we discussed how RDF triples may eventually transition into:

graph databases

such as:

Neo4j.

However, Chapter (13) now reveals an important truth:

triples alone are not enough.

A graph database fundamentally stores:

relationships.

But relationships themselves without semantic governance quickly become:

disconnected meaning.

Imagine importing ontology relationships into Neo4j without preserving "Domain" and "Range" logic.

Suddenly: anything may connect to anything!

Over time: semantic ambiguity increases.

- Graph traversal becomes noisy.
- Impact analysis becomes misleading.
- Enterprise architects lose confidence in the graph.

This challenge becomes increasingly serious in large organizations.

Because enterprise ecosystems contain:

- objectives
- policies
- business capabilities
- applications
- processes
- risks
- regulations
- infrastructure components

- partners

Without semantic discipline:

graph sprawl emerges.

Domain and Range therefore introduce an important idea:

**semantic filtering.**

Relationships become: intentional.

Queries become: explainable.

Dependencies become: trustworthy.

This matters enormously for:

AI-ready enterprise architecture.

Because AI systems require more than:

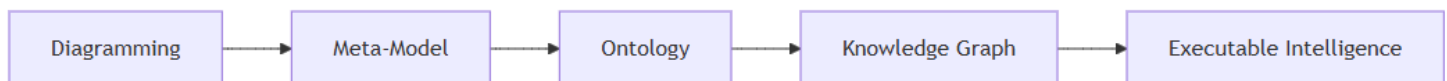
connected information.

They require:

**high-quality structured meaning.**

This is precisely where ontology provides value beyond traditional architecture repositories.

Within the EKA roadmap:



Chapter (13) strengthens the transition between:

ontology

and:

trusted knowledge graph.

Because meaningful intelligence requires:

meaningful boundaries.

## 13.10 Governance and Semantic Integrity

As ontology models become larger and increasingly interconnected, an important realization gradually emerges:

semantic freedom without governance creates semantic chaos.

At first, ontology engineering may feel deceptively simple in below steps:

1. Create classes.
2. Connect them through relationships
3. Add object properties.
4. Enable reasoning

However, as you progress deeper into semantic modeling, you may quickly discover an important truth:

ontology quality depends heavily on semantic discipline.

This chapter introduces one of the most important governance principles in ontology engineering:

relationships require boundaries.

Not every relationship should be allowed to exist!

And not every technically possible connection should be considered:

semantically meaningful!

This distinction matters enormously.

Because ontology systems increasingly influence:

- knowledge graph
- dependency tracing
- enterprise architecture analysis
- governance automation
- AI reasoning systems

Incorrect semantic assumptions therefore create:

incorrect intelligence.

Domain and Range exist precisely to reduce this risk.

They help ontology engineers formalize **semantic intent**.

Earlier in this chapter, you saw how:

```
hasTopping
```

and:

```
isToppingOf
```

maintain semantic correctness through carefully aligned Domains and Ranges.

Without these boundaries, ontology reasoners may infer:

```
logically incorrect classifications.
```

And because ontology reasoners trust:

```
semantic definitions,
```

mistakes may quietly and quickly propagate throughout the whole model.

This introduces an important lesson:

```
ontology mistakes often scale faster than traditional modeling mistakes.
```

Why?

Because inference multiplies impact.

One poorly designed semantic rule may generate hundreds of unintended conclusions.

This becomes especially important within enterprise-scale ontology.

Imagine an architecture ontology where:

```
supports object property
```

has overly broad semantic boundaries.

Suddenly:

```
Applications supports Regulations
```

or:

Infrastructure supports Business\_Strategy

may become semantically inferable.

Even if technically possible within the graph structure:

the meaning becomes questionable.

Ontology therefore requires:

**semantic governance!**

From the perspective of:

$$EKA = (K, R, \Theta, \Phi, \Gamma)$$

this chapter strongly reinforces:

**$\Gamma$  - Governance**

because governance fundamentally depends upon:

semantic correctness, not only technical correctness.

$\Gamma$  - Governance in EKA is not merely:

documentation standards.

Nor is it limited to:

architecture approval processes.

Instead, governance increasingly becomes:

**machine-enforceable semantic discipline.**

Ontology therefore evolves from:

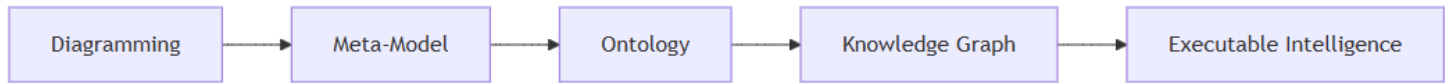
passive architecture knowledge

toward:

governed executable knowledge.



This represents an important maturity transition in the EKA roadmap.



Because enterprise intelligence becomes trustworthy only when:

- semantic quality becomes governable.

A useful mindset for ontology engineers is therefore:

- Model relationships conservatively**

- Expand carefully**

Good ontology engineering values:

- semantic precision

before:

- semantic flexibility.

Because trustworthy intelligence always depends upon:

- trustworthy semantics.**

## 13.11 Common Modeling Mistakes in Domain and Range

As learners begin applying Domain and Range in their own ontology projects, mistakes naturally emerge.

This is completely normal. In face, mistakes and unexpected reasoning results can provide valuable opportunities to deepen one's understanding of semantic behavior.

In fact, ontology engineering often advances through:

- experimentation, reasoning surprises, and semantic correction.

Because OWL reasoners - they are machines - interpret logic very precisely, even seemingly **small** modeling choices may create:

unexpectedly **large** semantic consequences.

The following are some of the most common mistakes ontology beginners encounter when working with:

Property Domain and Range.

Understanding them early will significantly improve ontology quality.

### 13.11.1 Overly Broad Domain Definitions

One of the most common beginner mistakes is assigning highly generic classes such as:

Thing

as the Domain of a property.

Technically, this works.

However, semantically:

it weakens meaning.

Consider the `Pizza.owl` property:

`hasTopping`

If its Domain is defined as:

Thing

rather than:

Pizza

the ontology would allow **virtually anything** to participate in the relationship.

Potentially:

`Restaurant hasTopping MushroomTopping`

or even:

`Country hasTopping CheeseTopping`

While logically allowed by the model:

semantically this becomes questionable.

Ontology therefore benefits from: semantic precision.

A useful engineering principle is:

define relationships as narrowly as business meaning reasonably permits.

This improves:

- reasoning quality
- semantic explainability
- governance reliability

## 13.11.2 Missing Range Definitions

Another common mistake occurs when ontology engineers define Domain but forgot to define Range.

This weakens semantic clarity.

Relationships now lack clear definition regarding:

what kinds of entities may receive meaning.

Ontology reasoners therefore possess less information for **inference**.

Within `Pizza.owl` ontology, if:

`hasTopping`

had no Range definition, the ontology would no longer clearly communicate:

what counts as a topping.

Reasoning quality declines.

Semantic trust weakens.

This issue becomes increasingly serious in:

enterprise knowledge graphs

where semantic ambiguity can easily spread across thousands of interconnected nodes.

A useful modeling habit is therefore:

define both sides of semantic meaning at the same time whenever possible.

Because every meaningful relationships contains:

a source

and:

a destination.

Mapping to the natural language: prevent just speaking half sentence.

### 13.11.3 Misunderstanding Multi-Domain and Multi-Range Logic

As discussed earlier in this chapter, ontology beginners often misunderstand multi-Domain and multi-Range configuration.

Many assume multiple semantic boundaries imply:

either/or logic (logical OR ).

However, OWL frequently interprets multiple Domains and Ranges through:

logical intersection (logical AND ).

This means ontology may unexpectedly infer that an entity belongs to:

multiple classes simultaneously.

For example:

If a property defines Domains:

- Pizza
- Restaurant

ontology may interpret usage as requiring something that is both:

Pizza **AND** Restaurant.

This frequently surprises beginners.

#### Modeling **OR** Ranges Correctly

If you need a property whose range can be **either** Business\_Process **or** Application:

- **Do not** add two `rdfs:range` axioms.
- **Do** use one of the patterns describes in **Section 13.5.2**: `owl:unionOf` , separate properties, or local restrictions.

A summary of the trade-offs is repeated here for convenience:

| Approach                                                      | Where to Use                                                                        |
|---------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <code>owl:unionOf range</code>                                | Semantically clean, reusable, tells reasoner the target can be one of several types |
| <b>Separate properties</b>                                    | Clearer semantics, easier governance, but more properties to maintain               |
| <b>Local restrictions</b><br>( <code>SubClassOf some</code> ) | Avoids global side effects; good for modular design                                 |

Choosing the right pattern depends on your ontology's scope, governance requirements, and the nature of the relationship.

Ontology engineers should therefore always:

|

validate assumptions through reasoning.

The reasoner often becomes: the best ontology teacher.

Unexpected inference frequently reveals:

|

**hidden** semantic misunderstandings.

### 13.11.4 Ignoring Inverse Property Alignment

Because Chapter (13) directly builds upon:

|

**Chapter (11) -- Inverse Properties**

another important modeling mistake occurs when inverse properties become:

|

semantically inconsistent.

For example:

Suppose `hasTopping` defines:

| Domain | Range        |
|--------|--------------|
| Pizza  | PizzaTopping |

But its inverse `isToppingOf` uses incompatible boundaries.

You can refer to one dedicated RDF file `pizza-ebook_13.11.4.rdf` , in which we assert the Domain and Range for these two object properties as below:

| Object Property          | Domain            | Range        |
|--------------------------|-------------------|--------------|
| <code>hasTopping</code>  | Pizza             | PizzaTopping |
| <code>isToppingOf</code> | <b>otherClass</b> | Pizza        |

Once synchronizing reasoner:

- `hasTopping` infers `otherClass` into Ranges:

The screenshot displays a reasoner interface with two main panes. The left pane, titled 'Object property hierarchy: hasTopping', shows a tree structure with 'owl:topObjectProperty' as the root, and 'hasTopping' and 'isToppingOf' as child properties. The right pane, titled 'Description: hasTopping', shows the configuration for the 'hasTopping' property. It includes sections for 'Equivalent To', 'SubProperty Of', 'Inverse Of', 'Domains (intersection)', and 'Ranges (intersection)'. The 'Inverse Of' section shows 'isToppingOf'. The 'Domains (intersection)' section shows 'Pizza'. The 'Ranges (intersection)' section shows 'PizzaTopping' and 'OtherClass'.

- `isToppingOf` infers `PizzaTopping` into Domains:

The screenshot shows a web ontology editor interface. On the left, under 'Object properties', the hierarchy is: owl:topObjectProperty > hasTopping > isToppingOf. The 'isToppingOf' property is selected. On the right, the 'Description: isToppingOf' panel shows various configuration options. The 'Inverse Of' section has 'hasTopping' selected. The 'Domains (intersection)' section lists 'OtherClass' and 'PizzaTopping'. The 'Ranges (intersection)' section lists 'Pizza'. The 'Annotations' section is empty.

Semantic inconsistency may emerge.

Reasoning behavior may become confusing.

Ontology quality weakens.

Michael's Exercise 11 quietly demonstrates an important best practice:

inverse properties should maintain logically mirrored semantic boundaries.

This preserves:

semantic coherence.

Relationships remain:

explainable

and:

reasoning stays trustworthy.

## 13.11.5 Treating Domain and Range Like Database Constraints

Another frequent misconception is treating Domain and Range as if they behave exactly like:

relational database constraints.

Traditional databases often reject invalid data.

While ontology behaves differently.

OWL reasoners primarily use:

Domain and Range for semantic interpretation.

For example:

Suppose ontology encounters in `Pizza.owl` :

`Pepperoni isToppingOf MySpecialPizza`

Even without explicit typing, the ontology reasoner may infer:

`Pepperoni is a PizzaTopping`

and:

`MySpecialPizza is a Pizza.`

The ontology did not reject the statement.

Instead:

it interpreted **meaning**.

This distinction is extremely important.

Ontology systems are fundamentally:

meaning-driven systems.

Not merely:

validation systems.

## 13.11.6 Over-Engineering Semantic Restrictions

Finally, ontology beginners occasionally become overly enthusiastic and begin introducing:

excessive semantic restrictions.

- Too many Domains.
- Too many Ranges.



- Too many property constraints.

Eventually, ontology becomes:

- difficult to understand,
- difficult to maintain, and
- difficult to explain.

Remember:

ontology exists to clarify meaning -- not to maximize complexity.

A practical ontology engineering principle is:

**simple semantics first, advanced semantics later.**

Even `Pizza.owl` intentionally introduces semantic sophistication:

progressively.

Good ontology design values:

clarity before cleverness!

## 13.12 Chapter Summary

In this chapter, you explored one of ontology engineering's most important semantic mechanisms:

**Property Domain and Range.**

Earlier chapters introduced `classes`, `object properties`, `inverse relationships` and `property characteristics`.

Chapter (13) extended this semantic journey by introducing:

**relationship boundaries.**

You discovered that:

Domain defines where a relationship originates,

while:

Range defines what kinds of concepts may receive the relationship.

Through Michael's Exercise 11 within `Pizza.owl` tutorial, you explored how:

inverse properties

and:

Domain/Range semantics

work together to preserve:

semantic consistency.

More importantly, you discovered a crucial ontology insight:

Domain and Range do not merely restrict meaning --

they also support:

**semantic inference.**

Ontology reasoners may automatically infer:

- class membership
- semantic classification
- relationship correctness

through carefully designed semantic boundaries.

The chapter also extended these ideas beyond `Pizza.owl` and explored how Domain and Range contribute toward:

- enterprise ontology,
- knowledge graphs, and
- **Executable Knowledge Architecture (EKA).**

From the perspective of:

$$EKA = (K, R, \Theta, \Phi, \Gamma)$$

this chapter particularly strengthened:

**$K$  - Knowledge Graph**

through semantic graph quality.

## ***R* - Reasoning & Rules**

through semantic inference,

and:

## **$\Gamma$ - Governance**

through semantic integrity.

Ultimately, Chapter (13) teaches an important engineering lesson:

meaningful intelligence depends on meaningful boundaries.

Because ontology becomes powerful not when:

everything connects to everything --

but when:

meaning is carefully governed.

This chapter was also refined through external technical review by **Timothy W. Cook**, whose feedback helped sharpen several aspects of the discussion, particularly the distinction between semantic inference and operational enforcement, the interpretation of domain inference in practical Protégé usage, and the role of governance within the EKA maturity model.

These observations are incorporated into the current edition as part of the continuing technical review of the book. They also provide useful guidance for the governance, validation, and executable-knowledge discussions developed in subsequent chapters.

## **13.13 Exercise 12 Transition -- From Domain and Range to Semantic Restrictions**

As Chapter (13) approaches its conclusion, the `Pizza.owl` tutorial quietly introduces an important conceptual transition through:

### **Exercise 12**

At first glance, you may perceive this as merely the beginning of the next chapter.

However, from an ontology engineering perspective:

Exercise 12 represents a major conceptual shift.

Up to this point in the learning journey, ontology modeling has primarily focused on **semantic structure**.

You created:

- classes
- subclass hierarchies
- object properties
- inverse relationships
- property characteristics
- Domain and Range boundaries

Ontology has therefore answered questions such as:

- What kinds of things exist?
- How are they related?
- What semantic boundaries govern relationships?

Exercise 12 now introduces a new question:

How do we describe what a class logically requires?

This distinction is subtle -- yet extremely important.

Consider the difference between the following statements, you now repeat the Domain and Range configuration for another property:

`hasBase`

This exercise demonstrates how semantic boundaries can be consistently applied across multiple properties while reinforcing inverse property reasoning.

Steps:

1. In the **Object Properties** tab, select `hasBase`.
2. Click the **Add (+)** icon next to **Domains (intersection)** in the Description view. From the ClassHierarchy tab, select `Pizza` as the domain.
3. Click the **Add (+)** icon next to **Ranges (intersection)** in the Description view. Select `PizzaBase` as the range.
4. Synchronize the reasoner.
5. Next, select `isBaseOf` — the inverse of `hasBase`. You will observe that the reasoner has automatically filled the Domain and Range, mirroring the inverse relationship.

This exercise reinforces several important points:

- **Consistency across inverse properties:** Domains and Ranges of inverse properties are automatically inferred, reducing manual effort.
- **Semantic governance:** Applying proper Domain and Range ensures relationships remain meaningful.
- **Practical reasoning:** Observing how the reasoner fills the inverse property demonstrates the interplay between semantic definitions and automatic inference.

## 13.14 Key Concepts

| Concept                    | Description                                                                                                         |
|----------------------------|---------------------------------------------------------------------------------------------------------------------|
| Property Domain            | Defines the class from which a relationship is expected to originate.                                               |
| Property Range             | Defines the class that is expected to receive the relationship.                                                     |
| Semantic Boundary          | Logical constraints that help preserve meaningful ontology relationships.                                           |
| Semantic Inference         | The process where ontology reasoners automatically infer meaning based on semantic rules.                           |
| Inverse Property Alignment | The practice of maintaining consistent Domain and Range semantics across inverse relationships.                     |
| Semantic Governance        | The discipline of preserving ontology quality through meaningful semantic constraints.                              |
| Knowledge Graph Integrity  | The maintenance of trustworthy graph relationships through semantic correctness.                                    |
| Executable Knowledge       | Knowledge that becomes machine-understandable, governable, and increasingly automatable through semantic reasoning. |

## 13.15 Protégé Skills Learned

By completing this chapter, you should now be able to:

- Configure Property Domain definitions in Protégé
- Configure Property Range definitions in Protégé
- Understand how Domain and Range affect inverse properties

- Observe semantic inference generated through reasoning
- Evaluate multi-Domain and multi-Range configurations
- Interpret reasoning outcomes caused by semantic boundaries
- Apply semantic governance thinking to ontology design
- Understand how ontology quality impacts Knowledge Graph implementation

## 13.16 Next Chapter Preview

In the next chapter (14), based on:

### **Video #14 -- Covering Pizza with Restrictions**

you will move beyond semantic boundaries and begin exploring:

### **OWL Restrictions.**

This represents an important transition in ontology maturity.

Instead of only defining "what kind of relationship exist", you will begin expressing "semantic rules about class behavior".

For example:

A pizza may be defined as:

having at least one topping,

or:

containing only vegetarian toppings.

Ontology modeling now becomes increasingly expressive!

You will begin learning how OWL transforms ontology from:

connected semantics

toward:

**logical semantic definitions.**

This chapter (14) also marks another important step toward the EKA vision of:

executable semantic intelligence.

Because restrictions introduce:

machine-interpretable business logic.

## 13.17 One more thing...

Throughout this chapter, we focused exclusively on **Object Properties** – the relationships that connect individuals to individuals.

However, semantic boundaries (Domain and Range) are not limited to Object Properties alone.

OWL also defines:

- **Data Properties** – connect individuals to literal values (e.g., strings, numbers, dates)
- **Annotation Properties** – add metadata to ontology entities (e.g., `rdfs:label` , `rdfs:comment` )

### Important

#### Clarifying "Annotations" vs. "Semantic Annotations"

A common source of confusion in modeling is informally using terms like "annotation" or "semantic annotation" to describe logical property characteristics (such as defining a property as `Transitive` or `inverseOf` ). To maintain strict technical precision, it is vital to distinguish between two completely different concepts:

- **Annotation Properties:** Used exclusively for human-readable metadata (e.g., `rdfs:label` , `rdfs:comment` ). Annotation properties are **completely invisible to the OWL reasoner**, their semantics are never enforced by the reasoner, and they cannot be used to define logical behaviors like `inverseOf` , `Functional` , or `Transitive` .
- **Object/Data Property Axioms & Characteristics:** Formal, logical definitions (e.g., `inverseOf` , `Functional` , `Transitive` , `Domain`, and `Range`) that define strict formal semantics interpreted enforced by the reasoner.

#### Domain and Range Across Property Types

Domain and Range work for Data Properties and Annotation Properties as well.

- For a **Data Property**, the Range is typically a concrete datatype (e.g., `xsd:string` , `xsd:integer` , `xsd:date` ) rather than an OWL class. For example, `hasCalories` (Data Property) can be modeled

with a Domain of `Pizza` and a Range of `xsd:integer`, communicating that only pizzas have a calorie count represented as an integer value.

- For an **Annotation Property**, Domain and Range can be used to restrict which kinds of ontology entities (classes, properties, individuals) may carry a given metadata annotation.

Applying clear Domain and Range boundaries for Data and Annotation Properties prevents semantic misuse and keeps your ontology consistent – whether you are modeling pizza, enterprise architecture, or a knowledge graph.

We will explore Data Properties and Annotation Properties in more detail in later chapters.

## Acknowledgments

The revisions to **Section 13.5.2** and **13.11.3** were motivated by an excellent question from **LaraAcuna** (GitHub Discussion #26: [https://github.com/yasenstar/protege\\_pizza/discussions/26](https://github.com/yasenstar/protege_pizza/discussions/26)), who asked about modeling `OR` ranges in OWL and highlighted the potential confusion around multi-range semantics. Her question prompted the addition of the `owl:unionOf` pattern, separate properties, and the local restriction alternative. Hence give me chance to summarize my understanding into the comparison table -- all of which strengthen this chapter's practical guidance for ontology engineers.

A special thanks to Michael for helping to clearly distinguish between **Annotations** and **Semantic Annotations**!

Last updated at: 2026-09-11